Basic Structure of a Procedural Program – Python

WI1102 Computational Thinking Teaching Team

Institut Teknologi Bandung © 2026

Objectives

Students will be able to:

- Explain the basic structure of a procedural program
- Explain data abstraction and basic data types
- Explain the meaning of and use variables and constants
- Explain and use:
  - assignment and input/output statements
  - expressions and operations related to data types
  - sequential actions

  correctly within a procedural program
- Create small programs using the Python language

Python

- A high-level programming language released by Guido van Rossum in 1991.
- Supports various programming paradigms; however, we will discuss only the procedural paradigm.
- Interpreters are available for a wide range of operating systems.
  - Uses indentation to define program blocks.
  - Case-sensitive: distinguishes between uppercase and lowercase letters.
- Python is a **strongly** and **dynamically** typed programming language.
  - **Strongly typed**: A variable's data type is determined by its initial assigned value, and subsequent operations must adhere to that type.
  - **Dynamically typed:** A variable's data type can be changed

Basic Structure of A Procedural Program

Program <ProgramTitle> { Program specification statements }
DECLARATIONS { Declarations of types, variables, constants, functions and procedures }
ALGORITHM { Algorithmic steps for problem solving }

This

Basic Structure of A Procedural Program in Python

```
# Program <ProgramTitle>
# Program specifications

# DECLARATIONS
# Written inside comments
# Declarations of types, variables, constants, functions and procedures

# ALGORITHM
# Algorithmic steps for problem solving
```

My First Program

- Write a program to print "Hello, World!" to the screen.

print is a command to print text to the screen/monitor

```
# Program HelloWorld
# Print Hello, World! to the screen

# DECLARATIONS
# not needed yet

# ALGORITHM
print('Hello, World!')
```

Input – Proses – Output




```
input (A)
input (B)
```

```
A ← A + B
```

```
output (A)
output (B)
```

pseudocode

Python

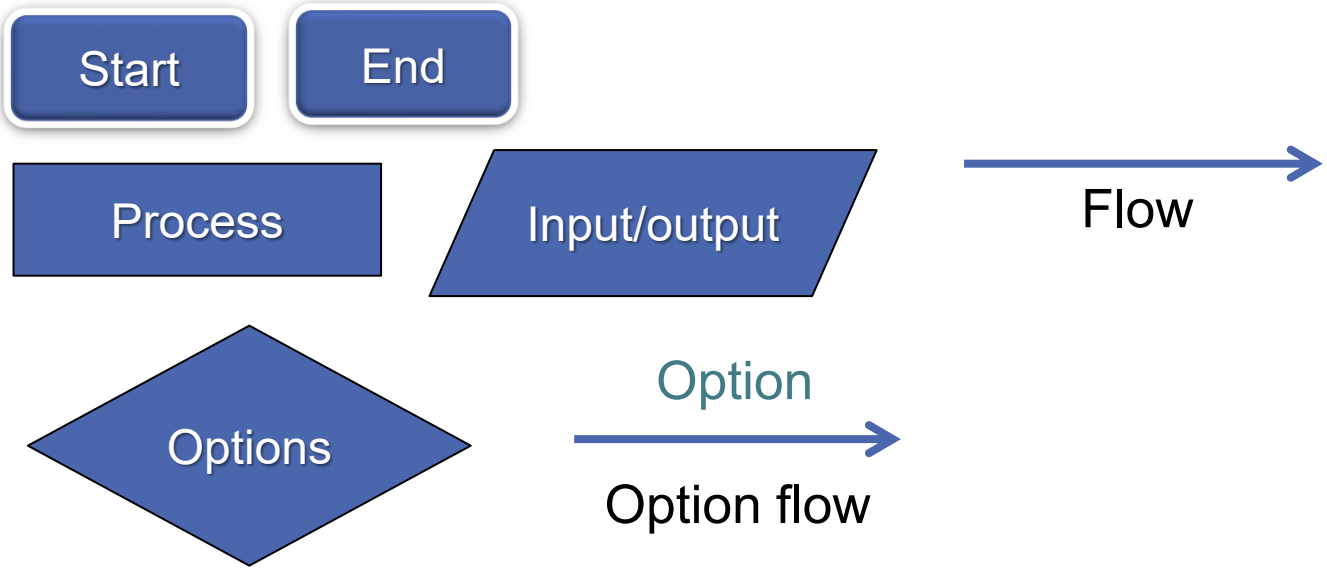
```
A = int(input(‘’))
B = int(input(‘’))
```

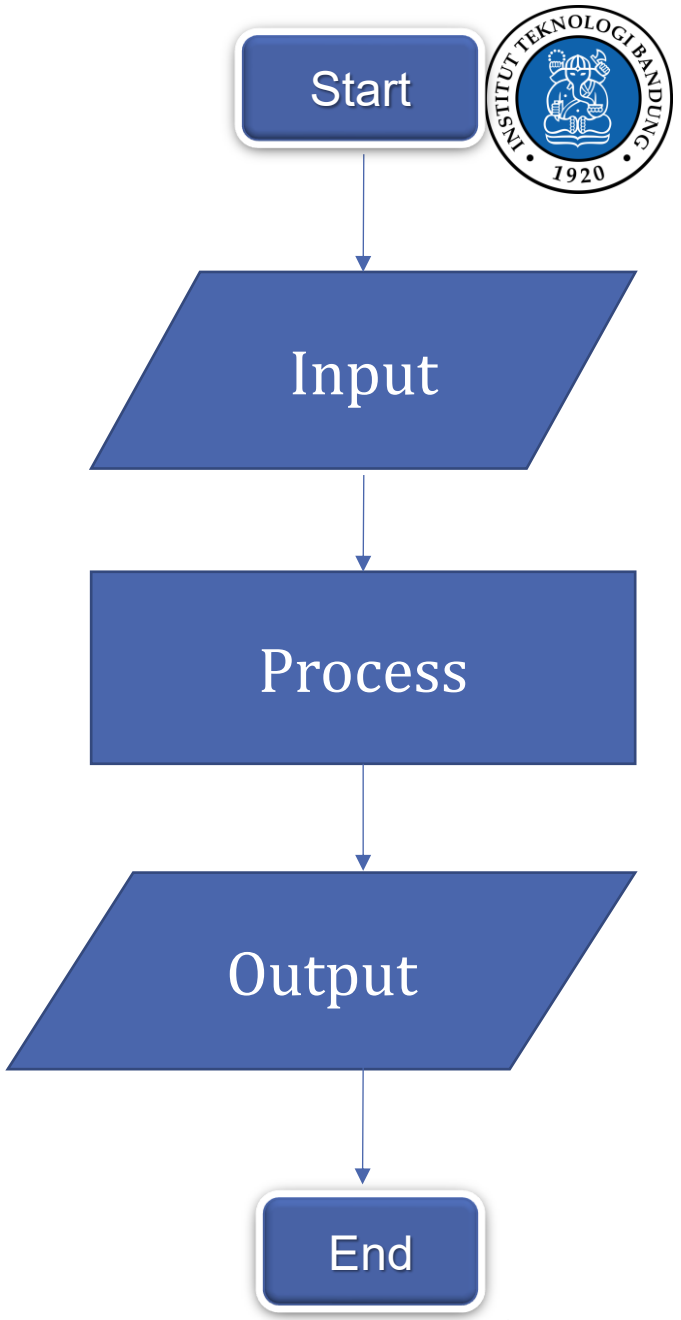
```
A = A + B
```

```
print(A)
print(B)
```

Flowcharts

- A **flowchart** is a visual representation of an algorithm or process that uses standardized symbols connected by arrows to show the flow of execution.
- Important symbols:





Pseudocode

- **Pseudocode** adalah deskripsi algoritma yang menggunakan bahasa sehari-hari, namun disusun agar menyerupai bahasa pemrograman yang disederhanakan *)
- Anggaplah pseudocode sebagai draf kasar untuk program Anda.
  - Pseudocode membantu kita merancang logika program sebelum mengimplementasikannya dalam bahasa pemrograman tertentu.
- Sering kali lebih dipilih daripada flowchart untuk perancangan program ketika Anda perlu menulis logika berbasis teks yang mendetail dengan cepat, menangani struktur bersarang yang kompleks, atau mengedit langkah-langkah yang menyerupai kode dengan mudah tanpa harus mengelola bentuk-bentuk grafis.
- Di kelas ini, kita akan menggunakan adaptasi **notasi algoritmik** yang dikembangkan oleh Inggriani Liem **) untuk menulis pseudocode.

*) https://www.codecademy.com/article/pseudocode-and-flowchart-complete-beginners-guide

**) Inggriani Liem: Draft Diktat Dasar Pemrograman (Bagian Pemrograman Prosedural), Edisi April 2007

Pseudocode

- **Pseudocode** is a description of an algorithm using everyday wording, but molded to appear similar to a simplified programming language *)
- Think of pseudocode as a rough draft for your program.
  - Pseudocode helps us outline our program's logic before implementing it in a specific programming language
- Often preferred over flowcharts for program design when you need to write detailed, text-based logic quickly, handle complex nested structures, or easily edit code-like steps without managing graphical shapes
- In this course, we will use adaptations of **algorithmic notation** developed by Inggriani Liem **) to write the pseudocodes

*) https://www.codecademy.com/article/pseudocode-and-flowchart-complete-beginners-guide
**) Inggriani Liem: Draft Diktat Dasar Pemrograman (Bagian Pemrograman Prosedural), Edisi April 2007

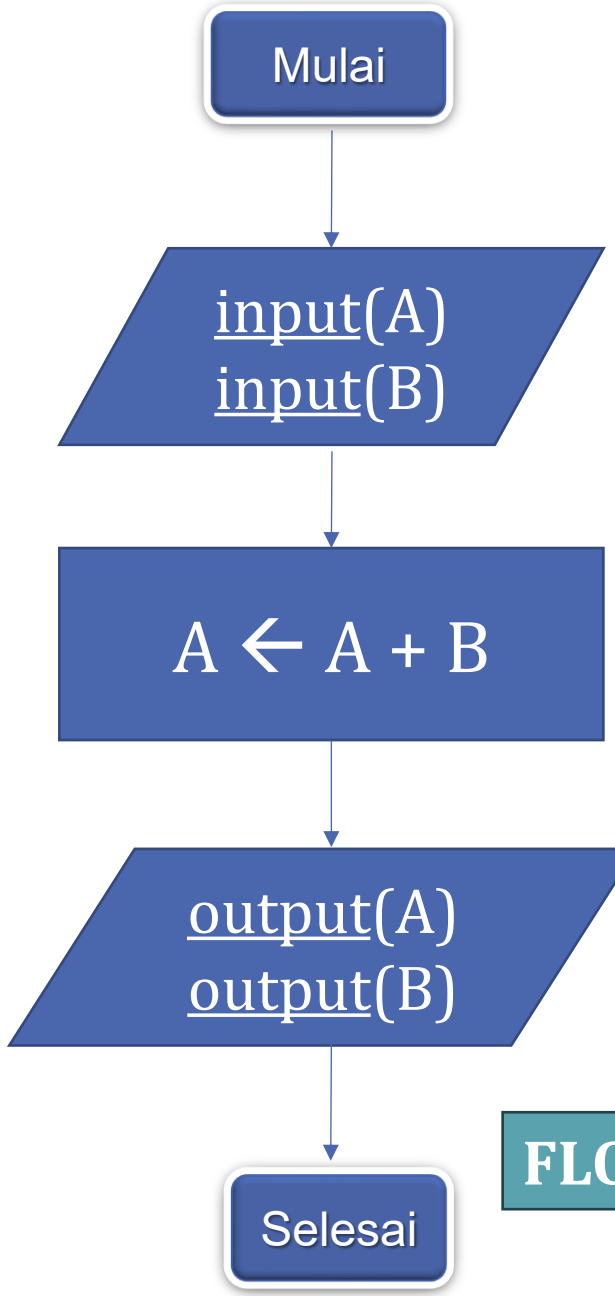
Basic Structure of A Program

PSEUDOCODE

Program Test
{ Calculate A + B }

DECLARATIONS
{ Deklarasi variable }
A, B : integer

ALGORITHM
input(A)
input(B)
A ← A + B
output(A)
output(B)



FLOWCHART

Example of Program in Python

```
# Program Test
# Specification: Calculate value of A and B

# DECLARATIONS
# A : int
# B : int

# ALGORITHM
A = int(input())  # input
B = int(input())

A = A + B         # process

print(A)          # output
print(B)
```

Program title and specification, written inside comments

DECLARATIONS: Declarations of variable A dan B

ALGORITHM: Input, Process, Output

The program code is inline with both pseudocode and flowchart from previous slide

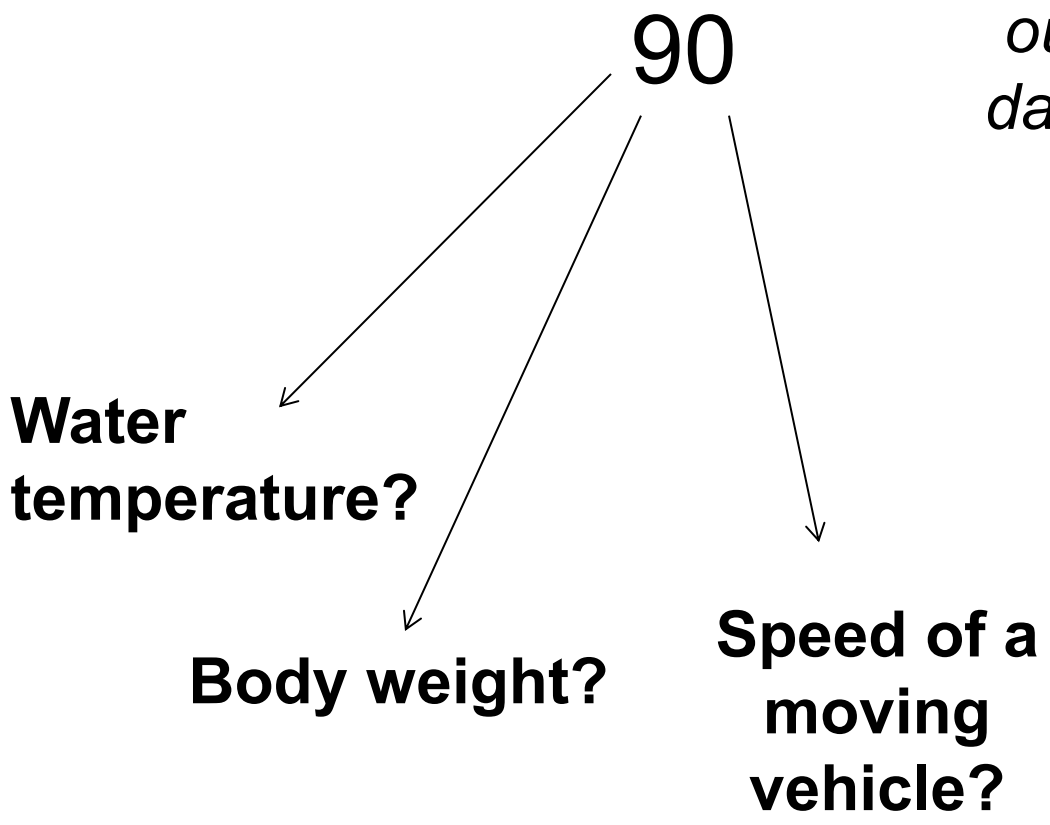
Comments

- In programming, a **comment** is a part of the program that is not executed.
  - This section is used solely to provide explanations for a step or formula, or simply to add a note
- In Python, comments are usually written on a per-line basis, starting with #
- Example:

```
# This is a comment
```

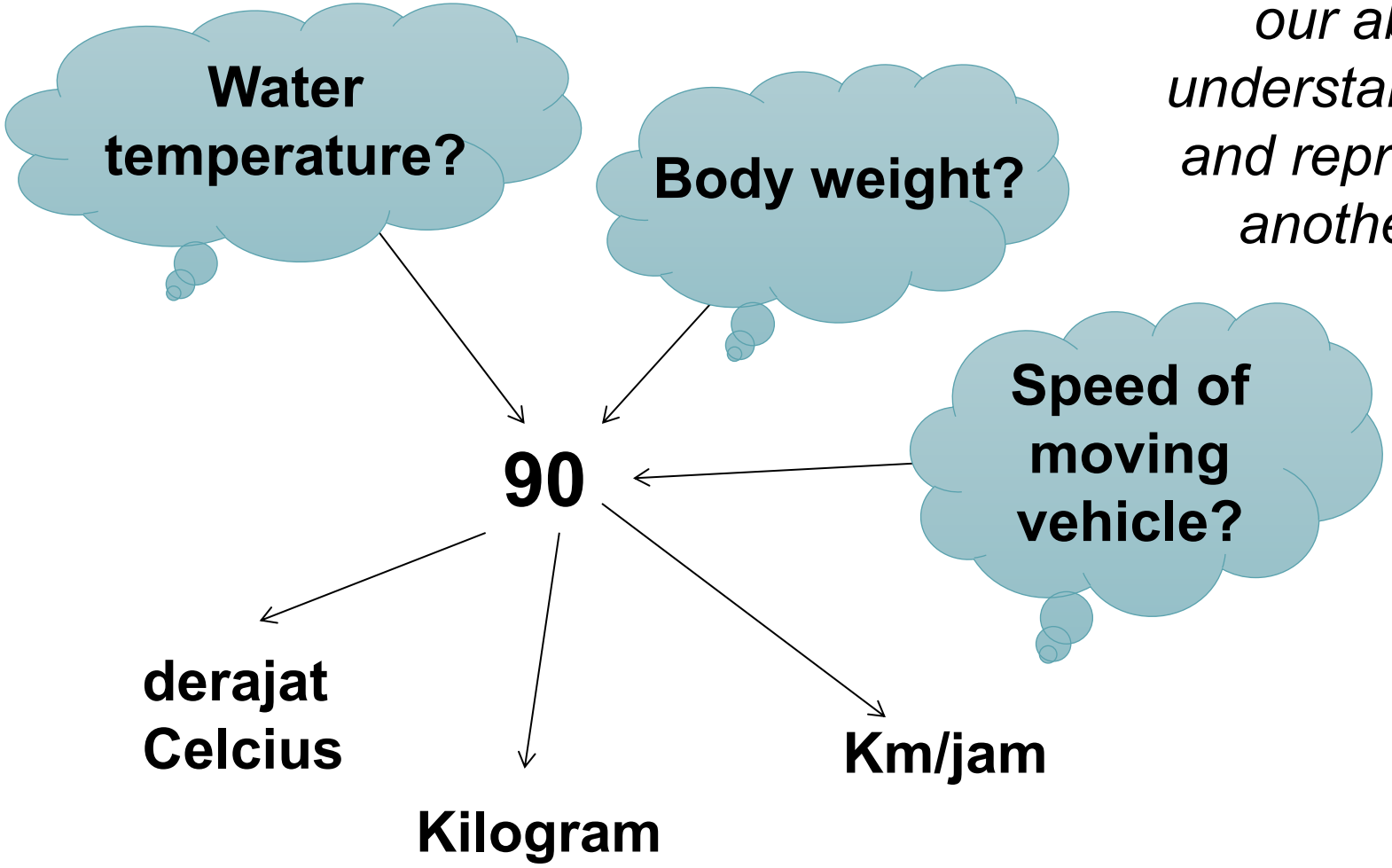
Data

Data Abstraction



our ability to interpret data within the context of the problem

Data Abstraction Problem



our ability to understand context and represent it in another form.

Value of Data

- Each data has a **set of values** allowed for it, for examples:
  - A**ge** of a person must be a positive integer (integer > 0)
  - **Name** of a person must be made of a sequence of letters (may contain numbers)
- Data may have **range** of values
  - Age of a person is between 1 and 100 (assuming a person's age cannot be more than 100 years)
  - The length of the name of person is between 1 and 50 (assuming no names longer than 50 letters)

DECLARATIONS

- The **DECLARATIONS** part of a program is used to declare "names" used within the program: names of data types, variables, constants, functions & procedure
- **Variable names** represent the data used in the program.
- Each variable has a set of values allowed for it → called **the data type**
- Examples of data types:
  - Integer: set of negative, 0, and positive integers
  - Boolean: comprises of only two values: true, false

Basic/Primitive Data Types

- Basic/primitive data types are provided by programming language

Python	Values
Bool	Boolean values: **True**; **False**
Numbers	Numeric values. Types of numeric values: • **int** : [signed] integer (+/-). Examples: 1; -144; 999; 0 • **float** : floating point (real). Examples: 3.14; 4.01E+1 • **complex** numbers → will not be used in this class
Char	Character: a single letter, marked by single quote. Examples: 'A'; '#'; 'b'
String	Set/sequence of characters/letters, marked by single or double quotes. Examples: 'xcxcx'

Examples of variables and their data types

- Age → Integer, examples: 25; 44; 35
- City → String, examples: “Jakarta”; “Bandung”
- Name → String, examples: “Budi”; “Ali”
- Temperature → Integer atau float, examples: 37.5; 100
- Area → Integer atau float, examples: 400; 43.5
- Body Weight → Integer atau float, examples: 60.5; 75
- Student No. → Integer atau string?, example: 15812001

Variable (1)

- **A variable** is a named component of a program code, used to store a value within a declared data type
- Represents data of something in the real world that want to be processed in the program
- Examples:
  - **weight**: represents the weight of a student in kg, data type: integer
  - **sum_weight** : represents the sum of the weight of all students, data type: integer
  - **mark**: represents course mark of student, data type: positive floating point/real number
- How to use a variable in a program:
  - Declare the variable: name and data type (in the DECLARATIONS part)
  - Value initialization and manipulation (in the ALGORITHM part)

Variable (2)

- Example of the declaration and initialization of a variable
  - In Python, write the declaration of the data type of a variable within DECLARATIONS part (in comments) → serves as important documentation for programmers

Python

```
# DECLARATIONS
# i : int
# A : int

# ALGORITHM
...
i = 100
A = i * 50
....
```

Create a Correct and Good Variable Names

- The name of a variable must be started with a letter and then followed with combination of letters and numbers. No other characters allowed except for underscore (_)
  - Should not be separated with space(s)
- Make a name that can be understood properly and unambiguous, preferably simple and short
  - For example: **sum** is commonly used to represent the result of number summarization, therefore must be of numeric data types (integer/real). It will be confusing otherwise, the type String or Boolean for example.
- Python is a ***case sensitive*** language: upper and lowercase characters are treated as a different characters
  - Example: **Weight** and **weight** are two different names

Assignment and Input/Output

Assigning Value of A Variable

- A known variable defined within a particular data types must be given **a value** to be used in a program
- Instruction to assign a value to a variable
  - Direct value assignment → ***assignment***, using the operator =
    - Example: **A = 10**
  - Provided via input device, using **input** instruction
    - examples: **A = input()**

Assignment

- General format: <left-hand-side> = <right-hand-side>
- <left-hand-side> must be a **variable**
- <right-hand-side> can be a value/expression/other variable [which has been assigned a value]

Python

```
<left-hand-side> = <right-hand-side>
```

```
Examples:
i = 10
Nama = “Maya”
X = i + 10
```

The value of X is assigned via an expression

Input

- **Input**: assignment of the value of a variable via input device, for example: keyboard (read user input)
- Command in Python: <var> = **input(‘<prompt>’)**
  - <var> is the variable
  - Replace <prompt> with a input instruction sentence that can be understood by the program user or leave it empty
  - The values entered via the device input must be of the same data type as the variable
- Examples:

```
A = input()                                  # A must be a string
B = input(‘Enter a number =')                # B must be a string
C = int(input())                             # C must be an integer
D = float(input('Masukkan angka =’))         # D must be a float/real number
```

Type checking: ensuring the value entered is of the same type of the variable

Type Conversion

Several important type conversion functions in Python

No.	Function & Description
1	**int(x)** Converts x into an integer
2	**float(x)** Converts x into a floating point (real)
3	**str(x)** Converts x into its string representation
4	**chr(x)** Converts an integer x into a character
5	**ord(x)** Converts a character Mengkonversi sebuah character x menjadi nilai integernya

Output

- **Output**: write/print the value of a variable/constant/results of expression to the output device, for example: monitor/screen
- Command in Python: **print**
- Examples:

```
print(A)                          # print the content of A to the screen
print('Hello')                    # print Hello to the screen
print(A * 4)                      # print the result of A * 4
print("Hello World!" + str(a))    # print Hello World! <value of a>
```

str(a) converts the value of variable A (can be a non-string variable into a string. + is a concatenation operator for strings.

Exercise

- Determine for each line of the following Python code (numbers are written within comments), whether the assignment is correct or not.
- If it is not correct, provide explanation.

```
# Program Exercise
# Exercise for data types and assignment

# DECLARATIONS
# IA : int
# FA, FB : float
# SA, SB : string
# BA : bool
# CA, CB : char

# ALGORITHM
IA = 10           # (1)
FA = 3.45         # (2)
FB = 4.567        # (3)
FB = IA           # (4)

SA = "ITBJAYA"    # (5)
SA = SB           # (6)

CA = 'C'          # (7)
CA = "MAJUTERUS" # (8)

BA = True         # (9)
BA = "#"          # (10)
```

Expressions

Expressions

- An **expression** is a combination of one or more variables, constants, operators, and functions that is meaningful according to the rules of a programming language and yields a value of a specific type.
- An **operator** is a standard function provided by a programming language to perform basic operations such as arithmetic, logical, and relational calculations.
- General structure of a [binary] expression:

```
<operand1> <operator> <operand2>
```

- The result of the operation depends on the data types of the operands.
- Operands can be values, variables, constants, or other expressions.

Types of Expressions

- Types of expressions based on operator arity:
  - **Binary expression**: fundamentally an operation with two operands
    - Examples: A **+** 5
  - **Unary expression**: fundamentally an operation with one operand
    - Examples: **not** (found)
- Types of expressions based on the resulting data type:
  - **Arithmetic** expression: operands are numeric (int/float) and result in a numeric value
  - **Relational** expression: operands are numeric (int/float) and result in a boolean/logical value
  - **Logical** expression: operands are boolean/logical and result in a boolean/logical value

Basic Data Type Operators (1)
Arithmetic Operators

If **a = 10** and **b = 21**, then:

Operator	Description	Operand Tyoe	Example
+	**Addition:** adds the values of the two operands	int, float	a + b = 31
-	**Subtraction:** subtracts the value of the right operand from the value of the left operand.	int, float	a – b = -11
*	**Multiplication:** multiplies the values of the two operands	int, float	a * b = 210
//	**Integer division:** If the operands are integers, the result of the operation is integer division.	int	b // a = 2
/	**Real division:** If the operands are floats, the result of the operation is floating-point division.	int, float	b / a = 2.1
%	**Modulo:** the remainder of integer division	int	b % a = 1
**	**Exponentiation:** raise the left operand to the power of the right operand.	int, float	10 ** 21 = 10^{21}

Basic Data Type Operators (2)
Relational Operators

If **a = 10** and **b = 21**, then:

Operator	Description	Operand Type	Example
==	If the values of both operands are the same, it returns true (does not apply to real numbers)	int, char, string, bool	(a == b) menghasilkan false
!=	If the values of the two operands are not equal, it yields true.	int, float, char, string, bool	(a != b) menghasilkan true
>	If the value of the left operand is greater than the right operand, it evaluates to true.	int, float, char, string	(a > b) menghasilkan false
<	If the value of the left operand is less than the right operand, it evaluates to true.	int, float, char, string	(a < b) menghasilkan true
>=	If the value of the left operand is greater than or equal to the right operand, it evaluates to true.	int, float, char, string	(a >= b) menghasilkan false
<=	If the value of the left operand is less than or equal to the right operand, it evaluates to true.	int, float, char, string	(a <= b) menghasilkan true

Basic Data Type Operators (3) Logical Operator

If **a = true** and **b = false**, then:

Operator	Description	Operand Type	Example
and	**AND**: If both operands are true, the result is true.	boolean	(a and b) equals to false
or	**OR:** If at least one of the two operands is true, the result is true.	boolean	(a or b) equals to false
not	**NOT**: Invert the logical value of the operand.	boolean	not(a) equals to false

Basic Data Type Operators (4)
Assignment Operators

If **a = 10** and **b = 21**, then:

Operator	Description	Type Operand	Examples
<op>=	<op> is + - * / % Shortens operation A = A <op> B into A <op>= B	int, float	a+=b; then a = 31 (equivalent to a = a + b) a*=b; then a = 210 (equivalent to a = a * b)

Sequential Actions

The ALGORITHM Part of A Program

- The part of a program in the form of algorithmic text containing instructions or action calls.
- Such algorithmic text can consist of:
  - Basic commands: Input/Output, assignment
  - Sequential commands
  - Case analysis/conditionals
  - Repetition... etc.
- In Python, each instruction is written on a separate line.
  - If more than one instruction is written on a single line, the instructions are separated by semicolons (;).
  - Examples: name = input(); print(name)

Sequential Action

- **Sequential action**: a series of primitive instructions and/or actions to be carried out (executed) by a computer in the order in which they are written.
- Each action alters the state of the program.
  - Therefore, every sequential action must have a beginning and an end.
  - In other words, a program must start and, at some point, must end.
- Instructions are written in a specific sequence, line by line.
- Note that:
  - there are programs whose behavior changes if the order of instruction lines is altered,
  - while there are others whose behavior remains unchanged if the order of instruction lines is altered.

The order of instructions does not change the execution result…

```
# Program Test
# DECLARATIONS
# i : int
# x : float

# ALGORITHM
i = int(input())
x = 100.75

print(x)
print(i*2)
```



```
# Program Test
# DECLARATIONS
# i : int
# x : float

# ALGORITHM
x = 100.75
i = int(input())

print(x)
print(i*2)
```

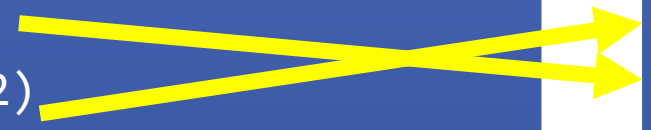
The execution result remains **unchanged**, even if the instruction order is altered.

The sequence of instructions changes the execution result…

```
# Program Test
# DECLARATIONS
# i : int
# x : float

# ALGORITHM
i = int(input())
x = 100.75

print(x)
print(i*2)
```



```
# Program Test
# DECLARATIONS
# i : int
# x : float

# ALGORITHM
i = int(input())
x = 100.75

print(i*2)
print(x)
```

The execution result **changes** because the order of instructions is altered.

Program Block (1)

- A series of instructions executed sequentially is grouped into a **program block**.
- In Python, a program block is marked by increasing indentation.
- A program block may contain another program block nested within it (an inner block).
- If instructions belong to the same block, the indentation must be consistent; otherwise, an error will occur.

Program Block (2)

```
a = int(input("Enter a number = "))
if (a > 50):
    print ("Hello World!")
    print ("bye")
else: # a <= 50
    print ("Hello Darling!")
    print ("bye bye")
```

OK!

```
a = int(input("Enter a number = "))
if (a > 50):
    print ("Hello World!")
  print ("bye")
else: # a <= 50
    print ("Hello Darling!")
      print ("bye bye")
```

Error!

Examples:

- Line 7 to 13 are in 1 block (*outer block*)
- Line 9 to 10 are in 1 blok (*inner block*)
- Line 12 to 13 are in 1 blok (*inner block*)

```
1   # Program Test
2
3   # DECLARATIONS
4   # a : int
5
6   # ALGORITHM
7   a = int(input("Enter a number = "))
8   if (a > 50):
9       print ("Hello World!")
10      print ("bye")
11  else: # a <= 50
12      print ("Hello Darling!")
13      print ("bye bye")
14
```

Command if-then-else… coming soon

Example 1. Pak Pit's Wheels

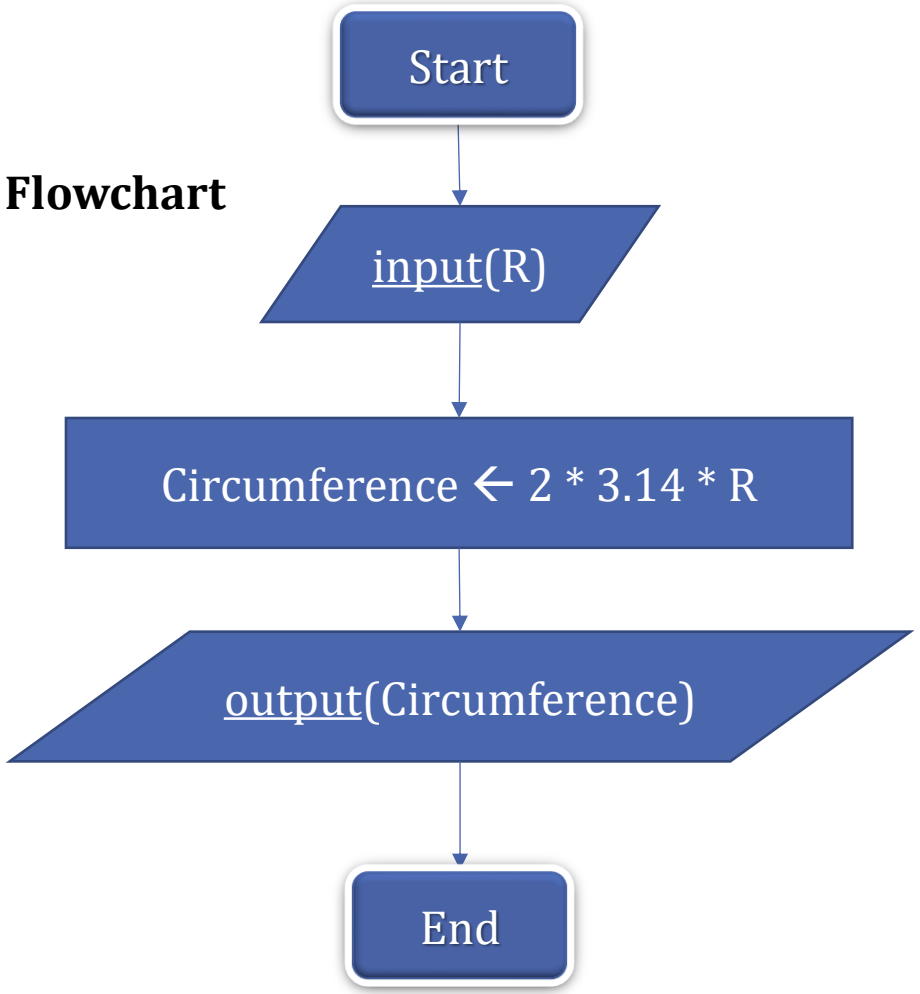
- Pak Pit, a bicycle repair shop owner, charges for each bicycle wheel he repairs based on the wheel's circumference.
- To do this, he measures the bicycle wheel's radius—the distance from the center of the wheel to its edge.
- Create a program that displays the calculated circumference of the circle based on the input radius value.
- The formula for calculating the circumference of a circle is: $2\pi r$
  - r is the length of the radius.

Example 1. Pseudocode + Flowchart

Pseudocode

```
input(R)
Circumference ← 2 * 3.14 * R
output(Circumference)
```

Flowchart



Examples 1. Python

```
# Program Circle_Circumference
# Calculates the circumference of a circle based on the input radius

# DECLARATIONS
# R : float
# Circumference : float

# ALGORITHM
R = float(input())

Circumference = 2 * 3.14 * R

print(Circumference)
```

Example 2. Average Height

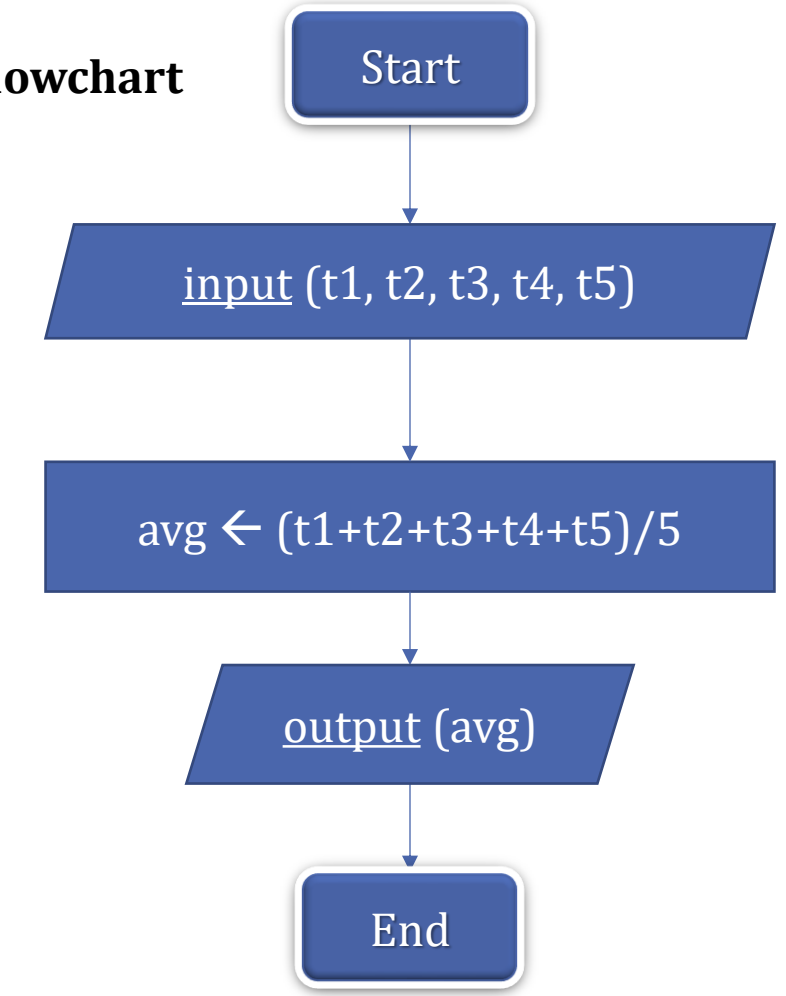
- The teacher selects five students to join the school basketball team. He wants to determine their average height.
- Create a program to calculate the average height of the five students.
  - The program will accept height data for the five students as input.
  - Then, the program will display the average height of the five students.

Example 2. Pseudocode + Flowchart

Pseudocode

```
input(t1, t2, t3, t4, t5)
avg ← (t1+t2+t3+t4+t5)/5
output(avg)
```

Flowchart



Examples 2. Python

```
# Program AverageHeight
# Accepts the heights of 5 students and calculates the average

# DECLARATIONS
# t1, t2, t3, t4, t5 : float
# avg : float

# ALGORITHM
t1 = float(input())
t2 = float(input())
t3 = float(input())
t4 = float(input())
t5 = float(input())

avg = (t1 + t2 + t3 + t4 + t5)/5

print (avg)
```

More Exercises

- For the following problems, practice creating:
  - A flowchart or pseudocode (choose one, or follow the instructor's requirement)
  - The corresponding Python program

Exercise 1. Calculate Distance

- In physics, distance (s) can be calculated based on speed (v) and travel time (t) using the formula: s = v * t.
- Write a program to calculate distance (in meters) based on input for speed (in m/s) and time (in seconds).

Exercise 2. Triangular Banners

- Mrs. Tuti is a well-known entrepreneur in her city who produces *umbul-umbul* (decorative vertical banners). She creates banners in various shapes, including triangles.
- For each triangular banner, Mrs. Tuti sets the price based on its area. To calculate the area, she needs the banner's height and base.
- Create a program that accepts the height and base as input and calculates the area of the triangular banner.
- Triangle area formula: area = ½ * base * height

Exercise 3. The Marble Shop

- A shop sells marbles. The following table shows the prices of the marbles based on their color:

Marble color	Price per marble (in hundred rupiah)
Red	10
Green	15
Yellow	20

- A child buys **m** red marbles, **h** green marbles, and **k** yellow marbles. Assume $m \geq 0$, $h \geq 0$, and $k \geq 0$.
- Calculate the total amount the child must pay.